PROJECT EXPLAINER · READ BEFORE YOU PRESENT

How The Rasaganga's catering site was built

This walks through the whole project the way you'd explain it out loud — what each piece is, why it's there, and what actually happens in the code when a customer books a thali. Real file names, real code, no filler.

`Python 3.12` `Django 6` `SQLite` `Bootstrap 5` `Pillow` `HTML / CSS`

01

What this project is

It's an online ordering system for **The Rasaganga**, a real South Indian vegetarian restaurant in Bellandur, Bangalore. There are two kinds of people who use it:

- **A customer** creates an account, looks through the menu or the catering packages, picks one for an event (date + number of guests), checks out, and can track whether the restaurant has confirmed the booking.
- **The restaurant's admin** logs into a separate dashboard to add or edit menu items and packages, and to work through incoming bookings — confirm them, reject them, or mark them completed.

Both sides are the *same website* — what a visitor can see and click depends on whether they're logged in, and whether their account is flagged as an admin. That single fact — one codebase, two experiences, gated by login — is worth being able to say in one sentence if someone asks "how does the admin part work?"

02

The toolkit, and why each piece is there

Nothing exotic — five moving parts, each doing one job. "When" below means *at what point* each one actually does something — some only matter while you're writing code, others run on literally every single page load.

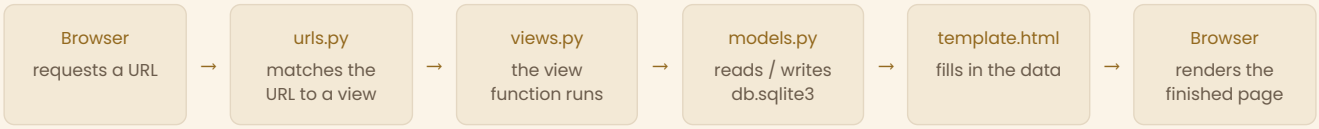
TOOL	WHAT IT ACTUALLY DOES HERE	WHEN IT RUNS
Python	The programming language everything else is written in.	Always — every request, every command.
Django	A Python *framework* — a large box of already-solved problems (user accounts, security, talking to the database, turning URLs into pages) so the project only had to write the parts specific to a catering business, not reinvent login forms and password hashing from scratch.	Every request — it's the process that receives the HTTP request and returns the response.
SQLite	The database. It's a single file (`db.sqlite3`) that stores every user, package, order, and review as rows in tables. Chosen because it needs zero setup — no separate database server to install — while still being a real, production-capable database for a project this size.	Any time a view reads or saves data — e.g. loading the package list, or saving a new Order at checkout.
Bootstrap 5	A CSS library that supplies ready-made building blocks — grids, buttons, forms, nav bars — so the pages are responsive (work on a phone) without hand-writing layout CSS for every screen size.	In the browser, once the HTML is downloaded — it's a CSS/JS file the page links to, not something the server runs.
Pillow	A Python library that Django uses behind the scenes to check and process uploaded images (package photos, menu photos) before saving them.	Only at upload time — when an admin submits a package/menu-item form with a new photo attached.

> **If asked "why not just plain HTML/PHP?"** — a catering site needs accounts, a cart, and an order database that PHP or plain HTML don't give you for free. Django supplies all three already wired together, correctly and securely, which is the entire reason to reach for a framework instead of writing it by hand.

03

The core idea Django is built on: MVT

Every single page on the site is produced the same three-step way. Django calls this pattern **Model – View – Template**, and once you can say this sentence, you can explain the whole backend: *a URL picks a view, the view asks a model for data, and hands that data to a template to turn into HTML.*



Model = the shape of the data (a Python class that becomes a database table). **View** = the logic (a Python function: what to fetch, what to check, what to save). **Template** = the HTML with blanks in it that the view fills in. Every feature in this project — login, the cart, the dashboard — is just this same loop, over and over, with different models/views/templates.

04

How the frontend works

The frontend is everything in the `frontend/` folder — it has no logic of its own. It's HTML pages with gaps in them, plus the CSS that styles those gaps.

Template inheritance

Every page shares the same navbar and footer, defined once in `frontend/templates/base.html`. Other templates don't repeat that HTML — they *extend* it and drop content into a named block:

`frontend/templates/base.html`

```
<!-- every page's shared skeleton -->
<nav class="navbar">...</nav>
<main>
    {% block content %}{% endblock %}
</main>
<footer>...</footer>
```

`frontend/templates/catering/menu.html`

```
{% extends 'base.html' %}

{% block content %}
  <!-- only the menu page's own content goes here -->
  <h1>Our Menu</h1>
  ...
{% endblock %}
```

That's why changing the navbar in one file (`base.html`) instantly updates every page on the site — they're all built on top of it.

Bootstrap does the layout, our CSS does the personality

Bootstrap's grid classes (`row`, `col-md-6`, and so on) handle responsive layout. `frontend/static/css/style.css` layers The Rasaganga's own look on top — the dark navbar, the gold accent color pulled from the restaurant's real signage, and the Playfair Display / Poppins fonts — as CSS custom

properties (`--brand-primary`, `--brand-gold`, etc.) so the whole palette can be changed by editing a handful of lines at the top of the file.

Forms render themselves

A form like the "add to cart" form isn't hand-typed as HTML — the template loops over the Python form object and lets each field draw its own input:

```
{% for field in form %}
    <label>{{ field.label }}</label>
    {{ field }}
    {% for error in field.errors %}
        <div class="text-danger">{{ error }}</div>
    {% endfor %}
{% endfor %}
```

This is why validation errors (like booking below a package's minimum guest count) appear automatically next to the right field, without the template needing to know the rule itself — that rule lives once, in the form's Python code (see the next section).

05

How the backend works

The backend is `backend/`. Four kinds of file do all the work:

Models — the shape of the data

Each model is a Python class; each field becomes a database column. Here's the real `Order` model — this single class is the entire reason an "order status" exists anywhere in the system:

`backend/orders/models.py`

```
class Order(models.Model):
    class Status(models.TextChoices):
        PENDING = 'PENDING', 'Pending Approval'
        CONFIRMED = 'CONFIRMED', 'Confirmed'
        REJECTED = 'REJECTED', 'Rejected'
        COMPLETED = 'COMPLETED', 'Completed'
        CANCELLED = 'CANCELLED', 'Cancelled'

    customer = models.ForeignKey('accounts.User', on_delete=models.CASCADE)
    status = models.CharField(max_length=10, choices=Status.choices, default=Status.PENDING)
    delivery_address = models.TextField()
    contact_phone = models.CharField(max_length=15)
    total_amount = models.DecimalField(max_digits=10, decimal_places=2)
```

Change this class, run one command (`makemigrations` then `migrate`), and Django rewrites the actual SQL table to match — no SQL was ever hand-written for this project.

Views — the logic

A view is a plain Python function that takes a web request and returns a web response. Here's the real `checkout` view, which is the single most important function in the ordering flow:

`backend/orders/views.py`

```python
@login_required
def checkout(request):
    cart = _get_cart(request.user)
    if not cart.items.exists():
        return redirect('orders:cart')

    if request.method == 'POST':
        form = CheckoutForm(request.POST)
        if form.is_valid():
            order = form.save(commit=False)
            order.customer = request.user
            order.save()
            for cart_item in cart.items.all():
                OrderItem.objects.create(order=order, package=cart_item.package, ...)
            order.recalculate_total()
            cart.items.all().delete()
            return redirect('orders:order_detail', pk=order.pk)
```

Every line here has a job: check the cart isn't empty, validate the address/phone form, create the `Order`, copy each cart line into a permanent `OrderItem` (so later menu-price changes never rewrite an old receipt), total it up, empty the cart, and send the customer to their new booking's page.

URLs — the routing table

Every address on the site is one line in a `urls.py` file, mapping a path to a view function by name:

`backend/catering_system/urls.py`

```python
urlpatterns = [
    path('admin/', admin.site.urls),
    path('accounts/', include('accounts.urls')),
    path('orders/', include('orders.urls')),
    path('', include('catering.urls')),
]
```

Each app owns its own slice of URLs (e.g. `orders/urls.py` defines `/orders/checkout/`), which is why the three apps can be worked on independently without their routes colliding.

Who's allowed to see what

Two small guards decide access on every request. `@login_required` sends anyone who isn't logged in to the login page first. A custom `@admin_required` decorator (in `accounts/decorators.py`) does the same but also checks `user.is_catering_admin` — the property on the `User` model that's `True` only for staff/admin accounts. Every dashboard view in the project starts with one of these two lines, which is the entire access-control system in one sentence.

06

A real booking, traced end to end

This is the version to walk someone through live — it shows every layer touching real code for one concrete action: **a customer books the Rasaganga Wedding Thali.**

1. **They click "Book This Package" on the package page.**
   catering/urls.py routes /packages/rasaganga-wedding-thali/ The `package_detail` view fetched the package by its slug and rendered `package_detail.html`, which linked to the booking form's URL.

2. **The booking form appears and gets submitted.**
   orders/views.py → add_to_cart() The `AddToCartForm` checks the event date isn't in the past and the guest count falls inside that package's min/max — both rules live in `orders/forms.py`, so the same check runs no matter which package is booked.

3 **A `CartItem` row is saved.**
orders/models.py → CartItem It's linked to the customer's `Cart`, the chosen `CateringPackage`, the event date, and the guest count. Nothing has been "ordered" yet — it's still just sitting in their cart.

4 **They go to checkout and submit their address + phone.**
orders/views.py → checkout() This is the function quoted above: it creates the `Order`, turns every `CartItem` into a permanent `OrderItem` (freezing today's price into the record), and empties the cart.

5 **The order sits as Pending Approval.**
orders/models.py → Order.Status.PENDING This is just the default value on the `status` field — nothing else happens until a human acts on it.

6 **The admin sees it in the dashboard and confirms it.**
orders/views.py → order_update_status() Guarded by `@admin_required`. It reads the new status from the submitted form and writes it straight onto the `Order` row.

7 **After the event, the admin marks it Completed.**
orders/views.py → add_review() Only now does a "Leave a review" button appear on the customer's order page — the view itself refuses to accept a review for any order that isn't `COMPLETED`, so it can't be gamed by reviewing before the food is even served.

> **The one idea worth remembering from this whole section:** nothing on this site "just happens" in a template. Every button either loads a URL (a GET) or submits a form (a POST); every POST is caught by a view function; every view function is the one and only place that's allowed to change the database. If someone asks "where's the logic for X", the answer is always: a function in some app's `views.py`.

07

The three apps, at a glance

Django organizes a project into small "apps" — each one owns a slice of the problem. This project has three, all listed in `INSTALLED_APPS` in `settings.py`:

accounts

Who's using the site, and as what role.

- `models.py` — custom User (+ role, phone, address)
- `views.py` — login, signup, profile
- `decorators.py` — @admin_required

catering

What's for sale — the catalog.

- `models.py` — Category, MenuItem, CateringPackage
- `views.py` — home, menu, packages + admin CRUD
- `management/commands/seed_data.py` — demo data loader

orders

Cart, checkout, bookings, reviews.

- `models.py` — Cart, Order, OrderItem, Review
- `views.py` — checkout flow + admin order queue

`catering` and `orders` both import from each other where it makes sense (an `OrderItem` points at a `CateringPackage`), but `accounts` is never imported by name — every other app just refers to "whichever model `settings.AUTH_USER_MODEL` points at", so the user model could be swapped without touching the other two apps.

08

The data model

How the real database tables connect to each other:

MODEL	RELATIONSHIP	MODEL
`User`	*has one*	`Cart`

MODEL	RELATIONSHIP	MODEL
`Cart`	*holds many*	`CartItem`
`User`	*places many*	`Order`
`Order`	*contains many*	`OrderItem`
`Order`	*can receive, once Completed*	`Review`
`Category`	*groups many*	`CateringPackage`
`CateringPackage`	*picked as*	`CartItem`
`CateringPackage`	*frozen into*	`OrderItem`
`CateringPackage`	*includes many*	`MenuItem`
`CateringPackage`	*receives*	`Review`

Two details are worth pointing out if asked: a `CartItem` always points *live* at a `CateringPackage` (so its price updates if the admin edits it), but an `OrderItem` copies the package's name and price onto itself at checkout — so a past receipt never silently changes just because the menu did later.

09

Running it, step by step

One server runs the whole thing — there's no separate frontend process to start, because Django builds the HTML itself before sending it to the browser. Every step below is a one-time setup step *except* the last one, which is what you run every time you want to work on or demo the site.

What you need first

- **Python 3.10+** installed and on your PATH (this project was built and tested on 3.12).
- A terminal open at the project's root folder (the one containing `backend/`, `frontend/`, and `venv/`).

1 **Create and activate a virtual environment.**

once, the first time only

```
python -m venv venv
venv\Scripts\activate
```

A virtual environment is a private, project-only copy of Python's package folder. Without it, installing Django here would install it *system-wide* instead, which risks clashing with whatever version any other Python project on the same machine needs. Activating it is what makes the terminal's `python` and `pip` commands point at that private copy instead of the system one.

2 **Install the two dependencies.**

once, or whenever requirements.txt changes

```
pip install -r backend\requirements.txt
```

Reads `requirements.txt` (just `Django` and `Pillow`, pinned to compatible versions) and downloads them into the virtual environment from PyPI, Python's public package index.

3 **Build the database.**

once, and again any time a model changes

```
python backend\manage.py migrate
```

Every model in every app's `models.py` has a matching "migration" file (auto-generated earlier by `makemigrations`) describing the SQL needed to create its table. `migrate` runs all of those against `db.sqlite3`, creating the file fresh if it doesn't exist yet. This is the one command that turns Python classes into an actual queryable database.

4 **Load realistic demo data.**

optional, safe to re-run any time

```
python backend\manage.py seed_data
```

Runs the custom command covered earlier — wipes and rebuilds The Rasaganga's real categories, menu, and packages, so the site never has to be demoed empty.

5 **Create an admin login.**

once

```
python backend\manage.py createsuperuser
```

Prompts for a username/email/password in the terminal and creates an account with `is_superuser=True` — which is what `user.is_catering_admin` checks for, so this account can reach `/dashboard/` immediately.

6 **Start the server.**

every time you want to use or demo the site

```
python backend\manage.py runserver
```

Starts Django's built-in development web server (not meant for production, but exactly right for building and demoing) and prints the address to open — normally `http://127.0.0.1:8000/`. Leave this terminal running; closing it stops the site. Press `Ctrl+C` in that terminal to stop it deliberately.

If something goes wrong

SYMPTOM	WHAT IT MEANS	FIX
"That port is already in use"	*Another program (often an old, still-running copy of this same server) is already on port 8000.*	Run `python backend\manage.py runserver 8001` and use that port instead, or close the other process.
"No such table" errors	*The database was never built, or a model changed since the last migrate.*	Run `python backend\manage.py migrate`.
Page has no styling at all	*Django can't find `frontend/static/`, usually from running commands outside the project root.*	Make sure the terminal's current folder is the repo root before running any `manage.py` command.

SYMPTOM	WHAT IT MEANS	FIX
Browser shows an unrelated site/error at localhost:8000	*A different, unrelated server is already answering that port.*	Use a different port (see above), or fully close the other server first.

For a live demo

- Sign up as a new customer, book a package, and show the order landing as *Pending* in "My Bookings".
- Log in as the admin and confirm that same booking from `/dashboard/` — then flip back to the customer tab and refresh to show the status changed live.
- `python backend\manage.py seed_data` resets the catalog back to a clean demo state at any time, if the data gets messy during a walkthrough.

10

Questions to expect — and short answers

Q Why Django instead of building it from scratch?

Accounts, security, and talking to a database are the same hard problems on every site. Django ships correct, tested solutions to all three, so the project's own code could focus on what's actually specific to a catering business.

Q Why SQLite, and what if you needed more scale?

SQLite needs no separate server — perfect for building and demoing. Because Django talks to the database only through the models, switching to MySQL/PostgreSQL later is a one-line change in `settings.py`, not a rewrite.

Q How are passwords kept safe?

They're never stored in readable form — Django hashes and salts every password automatically as part of its built-in auth system. This project never touches a raw password directly.

Q What stops a customer from opening the admin dashboard?

Every dashboard URL's view function starts with `@admin_required`, which checks `user.is_catering_admin` before the function's first line even runs. Guessing the URL isn't enough — it redirects straight to login.

Q Why split the project into "frontend" and "backend" folders if it's all one server?

Purely for readability — `backend/` is every file with logic in it, `frontend/` is every file that's pure presentation (HTML/CSS/images). `settings.py` is what tells Django to look for templates in the `frontend/` folder next door.

Q What would you add next?

A real payment gateway at checkout (bookings are currently confirmed manually, no online payment), and letting a customer build a custom order straight from the à la carte Menu page instead of only from pre-set packages.

11

Full code reference

Everything above explained the ideas. This is the reference to flip to if someone asks "show me the actual code" — every model, every form's validation, and every view not already quoted earlier, app by app, copied straight from the project.

accounts — who's using the site

accounts/models.py — the User model

Django ships a working `User` model already. This project extends it (rather than replacing it) to add a `role` and a one-property shortcut used everywhere access is checked:

```
class User(AbstractUser):
    class Role(models.TextChoices):
        CUSTOMER = 'CUSTOMER', 'Customer'
        ADMIN = 'ADMIN', 'Catering Admin'

    role = models.CharField(max_length=10, choices=Role.choices, default=Role.CUSTOMER)
    phone = models.CharField(max_length=15, blank=True)
    address = models.TextField(blank=True)

    @property
    def is_catering_admin(self):
        return self.role == self.Role.ADMIN or self.is_superuser
```

accounts/decorators.py — the admin gate, in full

The entire access-control rule for the dashboard is these two lines — it wraps a view and refuses to run it unless the visitor is logged in *and* is an admin:

```
def admin_required(view_func):
    return user_passes_test(
        lambda u: u.is_authenticated and u.is_catering_admin,
        login_url='accounts:login',
    )(view_func)
```

accounts/views.py — signup & profile

Signup logs the new customer straight in after saving them — no separate "verify your email" step:

```python
def signup(request):
    if request.method == 'POST':
        form = CustomerSignUpForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            return redirect('catering:home')
    else:
        form = CustomerSignUpForm()
    return render(request, 'accounts/signup.html', {'form': form})

@login_required
def profile(request):
    if request.method == 'POST':
        form = ProfileForm(request.POST, instance=request.user)
        if form.is_valid():
            form.save()
    else:
        form = ProfileForm(instance=request.user)
    return render(request, 'accounts/profile.html', {'form': form})
```

`CustomerSignUpForm` always forces `role = User.Role.CUSTOMER` on save — there is no signup field for role anywhere, which is *why* a stranger can never sign themselves up as an admin. Admin accounts only exist because a superuser created them (via `createsuperuser` or the Django admin).

catering — the catalog

`catering/models.py — MenuItem & CateringPackage`

A dish, and the bookable combo built from dishes:

```python
class MenuItem(models.Model):
    class FoodType(models.TextChoices):
        STARTER = 'STARTER', 'Starter'
        MAIN_COURSE = 'MAIN', 'Main Course'
        DESSERT = 'DESSERT', 'Dessert'
        BEVERAGE = 'BEVERAGE', 'Beverage'

    name = models.CharField(max_length=150)
    food_type = models.CharField(max_length=10, choices=FoodType.choices)
    is_veg = models.BooleanField(default=True)
    price = models.DecimalField(max_digits=7, decimal_places=2, null=True, blank=True)
    section = models.CharField(max_length=60, blank=True)  # groups the Menu page


class CateringPackage(models.Model):
    category = models.ForeignKey(Category, related_name='packages', on_delete=models.CASCADE)
    name = models.CharField(max_length=150)
    slug = models.SlugField(unique=True, blank=True)
    price_per_plate = models.DecimalField(max_digits=8, decimal_places=2)
    min_guests = models.PositiveIntegerField(default=20)
    max_guests = models.PositiveIntegerField(default=500)
    menu_items = models.ManyToManyField(MenuItem, related_name='packages', blank=True)
    is_active = models.BooleanField(default=True)

    @property
    def average_rating(self):
        agg = self.reviews.aggregate(avg=models.Avg('rating'))
        return round(agg['avg'], 1) if agg['avg'] else None
```

The `slug` (the URL-safe version of the name, like `rasaganga-wedding-thali`) is generated automatically the first time a package is saved — the admin never types one. `menu_items` is a **many-to-many** field: one package can include many dishes, and the same dish (say, Filter Coffee) can appear on many different packages.

`catering/views.py` — search & filter

The Packages page's category filter and search box are both just extra `.filter()` calls chained onto the same query, applied only if that field was actually submitted:

```python
def package_list(request):
    packages = CateringPackage.objects.filter(is_active=True)

    category_slug = request.GET.get('category')
    if category_slug:
        packages = packages.filter(category__slug=category_slug)

    query = request.GET.get('q')
    if query:
        packages = packages.filter(Q(name__icontains=query) | Q(description__icontains=query))

    paginator = Paginator(packages, 9)
    page_obj = paginator.get_page(request.GET.get('page'))
```

Nine packages per page, via Django's built-in `Paginator` — no custom pagination logic was written.

`catering/views.py` — the dashboard CRUD pattern

Every "add/edit" screen in the admin dashboard — packages, menu items, categories — follows this exact same shape. Once you've read one, you've read all nine:

```python
@admin_required
def package_create(request):
    if request.method == 'POST':
        form = CateringPackageForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            return redirect('catering:package_manage_list')
    else:
        form = CateringPackageForm()
    return render(request, 'catering/dashboard/package_form.html', {'form': form})
```

`request.FILES` is what carries the uploaded photo alongside the rest of the form — it's separate from `request.POST` because file uploads use a different encoding (`multipart/form-data`, set on the `<form>` tag in the template).

`catering/management/commands/seed_data.py` — the demo data loader

A custom command (run as `python manage.py seed_data`) that wipes the catalog and rebuilds it from The Rasaganga's real menu boards — this is how the site always has realistic data to demo instead of an empty database:

```python
class Command(BaseCommand):
    def handle(self, *args, **options):
        CateringPackage.objects.all().delete()
        MenuItem.objects.all().delete()
        Category.objects.all().delete()

        # ... re-creates every category, dish (with its real price), and
        # package, then links the right dishes to the right packages
        # via package.menu_items.set([...])
```

orders — cart, checkout, bookings

`orders/forms.py` — where the booking rules actually live

This is the validation mentioned back in the walkthrough — it's what stops someone booking a 400-guest wedding thali for one person, or for a date in the past:

```python
class AddToCartForm(forms.ModelForm):
    def __init__(self, *args, package=None, **kwargs):
        self.package = package
        super().__init__(*args, **kwargs)

    def clean_event_date(self):
        event_date = self.cleaned_data['event_date']
        if event_date < date.today():
            raise forms.ValidationError('Event date cannot be in the past.')
        return event_date

    def clean_guest_count(self):
        guest_count = self.cleaned_data['guest_count']
        if guest_count < self.package.min_guests:
            raise forms.ValidationError(f'Requires at least {self.package.min_guests} guests.')
        if guest_count > self.package.max_guests:
            raise forms.ValidationError(f'Supports at most {self.package.max_guests} guests.')
        return guest_count
```

Django calls any method named `clean_<fieldname>` automatically while validating — that naming convention is the only thing connecting these methods to their fields, nothing registers them by hand.

`orders/models.py — Cart & CartItem`

The cart total isn't a stored number — it's calculated fresh, every time, from whatever's currently in it:

```
class Cart(models.Model):
    customer = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)

    @property
    def total_amount(self):
        return sum((item.subtotal for item in self.items.all()), Decimal('0.00'))


class CartItem(models.Model):
    cart = models.ForeignKey(Cart, related_name='items', on_delete=models.CASCADE)
    package = models.ForeignKey(CateringPackage, on_delete=models.CASCADE)
    event_date = models.DateField()
    guest_count = models.PositiveIntegerField()

    @property
    def subtotal(self):
        return self.package.price_per_plate * self.guest_count
```

That's a **OneToOneField** — every customer has exactly one cart, created automatically the first time they add something (`Cart.objects.get_or_create(customer=request.user)` in the view), never explicitly by the user.

`orders/views.py — the admin's order queue`

The dashboard's booking list can be filtered by status, and updating a status is a one-field write guarded against invalid values:

```
@admin_required
def order_manage_list(request):
    orders = Order.objects.select_related('customer').prefetch_related('items')
    status_filter = request.GET.get('status')
    if status_filter:
        orders = orders.filter(status=status_filter)
    return render(request, 'orders/dashboard/order_manage_list.html', {'orders': orders, ...})

@admin_required
def order_update_status(request, pk):
    order = get_object_or_404(Order, pk=pk)
    if request.method == 'POST':
        new_status = request.POST.get('status')
        if new_status in Order.Status.values:
            order.status = new_status
            order.save(update_fields=['status'])
    return redirect('orders:order_manage_list')
```

`select_related` and `prefetch_related` are performance calls, not logic — they tell Django to fetch each order's customer and items in a couple of efficient queries up front, instead of running a fresh query per row while the template loops over the list (what's usually called the "N+1 query problem").

12

Configuration — settings.py, in full

Every Django project has exactly one settings file — here it's `backend/catering_system/settings.py`. It doesn't contain logic; it's a list of answers to questions Django asks on every single request ("where are the templates?", "which

database?", "who's the User model?"). Below is the real file, broken into the pieces that matter, each with what it controls and why that value was chosen.

`backend/catering_system/settings.py`

```
# BASE_DIR = the "backend" folder itself.
BASE_DIR = Path(__file__).resolve().parent.parent

# PROJECT_ROOT = one level up — the repo root, where "frontend" lives too.
PROJECT_ROOT = BASE_DIR.parent
FRONTEND_DIR = PROJECT_ROOT / 'frontend'
```

This pair of lines is the entire mechanism behind the backend/frontend folder split explained earlier — every other path in this file (templates, static files) is built from `FRONTEND_DIR` instead of `BASE_DIR`.

SETTING	REAL VALUE IN THIS PROJECT	WHAT IT CONTROLS
`INSTALLED_APPS`	*Django's 6 built-ins + accounts, catering, orders*	Which apps' models, templates, and admin registrations Django loads on startup.
`AUTH_USER_MODEL`	*'accounts.User'*	Tells Django to use the custom `User` model (with `role`) instead of its default one — must be set before the first migration ever runs.
`ROOT_URLCONF`	*'catering_system.urls'*	Which file Django starts reading URL patterns from on every request.
`TEMPLATES[0]['DIRS']`	*[FRONTEND_DIR / 'templates']*	Where `render()` looks for `.html` files — this is what makes `frontend/templates/` work at all.
`DATABASES`	*SQLite, file at BASE_DIR / 'db.sqlite3'*	Which database engine and file every model read/write goes to.
`STATICFILES_DIRS`	*[FRONTEND_DIR / 'static']*	Where `{% static %}` tags in templates resolve CSS/image files from.
`MEDIA_ROOT / MEDIA_URL`	*backend/media/ · /media/*	Where uploaded package/menu photos are saved on disk, and the URL prefix they're served back from.
`LOGIN_URL`	*'accounts:login'*	Where `@login_required` sends a signed-out visitor.
`LOGIN_REDIRECT_URL`	*'catering:home'*	Where a successful login sends the customer next.
`EMAIL_BACKEND`	*console backend*	Any email Django would send (e.g. password reset) is printed to the terminal instead of actually sent — fine for development, would be swapped for a real SMTP backend in production.
`DEBUG`	*True*	Shows Django's detailed error pages with full tracebacks. Would be set to `False` before ever deploying this publicly.

Why this file is safe to explain openly: the one setting that's normally sensitive — `SECRET_KEY` — is left at Django's auto-generated development default here on purpose. In a real deployment it would be moved out of this file entirely (into an environment variable) so it's never committed to version control.

Every URL route in the project

Three files, one per app, each imported into the root `catering_system/urls.py` shown earlier. Every clickable link and every form action anywhere on the site resolves to one row in one of these three tables.

`accounts/urls.py — 4 routes`

URL	VIEW	WHAT HAPPENS
`/accounts/signup/`	*signup*	Create a customer account, log them in immediately.
`/accounts/login/`	*CateringLoginView*	Django's built-in login form, styled to match the site.
`/accounts/logout/`	*CateringLogoutView*	Ends the session (POST-only, per Django 5+'s security default).
`/accounts/profile/`	*profile*	Edit name, email, phone, address.

`catering/urls.py — 15 routes`

URL	VIEW	WHAT HAPPENS
`/`	*home*	Landing page — hero, occasion cards, featured packages.
`/packages/`	*package_list*	All catering packages, filterable by category, searchable.
`/packages/<slug>/`	*package_detail*	One package's price, menu, and reviews.
`/menu/`	*menu*	The full à la carte menu, grouped by section.
`/dashboard/`	*dashboard_home*	Admin-only overview (package/category/menu-item counts).
`/dashboard/packages/…`	*4 views*	List, add, edit, delete catering packages.
`/dashboard/menu-items/…`	*4 views*	List, add, edit, delete menu items.
`/dashboard/categories/…`	*4 views*	List, add, edit, delete occasion categories.

`orders/urls.py — 10 routes`

URL	VIEW	WHAT HAPPENS
`/orders/cart/`	*cart_view*	Shows the signed-in customer's current cart.
`/orders/cart/add/<slug>/`	*add_to_cart*	The booking form for one package (event date, guests).
`/orders/cart/remove/<pk>/`	*cart_item_remove*	Deletes one line from the cart.
`/orders/checkout/`	*checkout*	Address + phone form → creates the real Order.
`/orders/my-orders/`	*my_orders*	A customer's booking history.
`/orders/my-orders/<pk>/`	*order_detail*	One booking's full detail and status.
`/orders/my-orders/<pk>/cancel/`	*order_cancel*	Cancels a still-Pending booking.
`/orders/my-orders/<pk>/review/<pk>/`	*add_review*	Star rating + comment, only once Completed.

URL	VIEW	WHAT HAPPENS
`/orders/dashboard/orders/`	*order_manage_list*	Admin's booking queue, filterable by status.
`/orders/dashboard/orders/<pk>/update-status/`	*order_update_status*	Admin moves a booking to its next status.

Every `name=...` on these routes (e.g. `name='checkout'`) is what lets a template write `{% url 'orders:checkout' %}` instead of hard-coding `/orders/checkout/` — if the URL path ever changed, every template referencing it by name would still work without any edits.

14

Admin panel setup (admin.py)

Separate from the custom `/dashboard/` built for this project, every Django project also gets a free, built-in admin panel at `/admin/`. Each app's `admin.py` just registers which of its models should be manageable there, and how — this is configuration, not application logic, so it's usually only a handful of lines.

`accounts/admin.py`

```
@admin.register(User)
class CateringUserAdmin(UserAdmin):
    fieldsets = UserAdmin.fieldsets + (('Catering Info', {'fields': ('role', 'phone', 'address')}),)
    list_display = ('username', 'email', 'role', 'is_staff', 'is_active')
```

Extends Django's own `UserAdmin` rather than replacing it — so the admin panel still gets Django's built-in password-change form and permissions UI, with `role`/`phone`/`address` added as one extra section.

`catering/admin.py`

```
@admin.register(CateringPackage)
class CateringPackageAdmin(admin.ModelAdmin):
    list_display = ('name', 'category', 'price_per_plate', 'min_guests', 'max_guests', 'is_active')
    list_filter = ('category', 'is_active')
    search_fields = ('name',)
    filter_horizontal = ('menu_items',)
```

`filter_horizontal` turns the many-to-many `menu_items` field into a searchable two-column picker instead of an unusable giant multi-select box — the one line that matters most here for actual usability.

`orders/admin.py`

```
class OrderItemInline(admin.TabularInline):
    model = OrderItem
    extra = 0

@admin.register(Order)
class OrderAdmin(admin.ModelAdmin):
    list_display = ('id', 'customer', 'status', 'total_amount', 'created_at')
    inlines = [OrderItemInline]
```

The `inlines` line is what lets an admin see every `OrderItem` belonging to an `Order` right there on the same page, instead of hunting through a separate OrderItem list — Django admin's way of showing a one-to-many relationship inline.

Why two admin surfaces exist: `/admin/` (this section) is a generic, auto-generated tool meant for the developer/superuser — useful, but it looks and feels like raw Django. `/dashboard/` (covered earlier) is hand-built with the site's own Bootstrap styling specifically for restaurant staff to use day-to-day. Most real Django projects keep both, for exactly this reason.

Every code sample above is copied verbatim from the project's own files — nothing here is simplified or invented for the explanation.